



Universidad Nacional Autónoma de México
Facultad de Ciencias
Redes neuronales
Profesor: Víctor Germán Mijangos de la Cruz[0.3cm]
Integrantes:
Martínez Marcelo Ingrid Aylen - 320179884
Pérez Evaristo Eris - 320287828
Ramírez Venegas Alexa Paola - 320111677



8 de Diciembre de 2025

Proyecto Final: Identificador de firmas falsas mediante Redes Neuronales Siamesas

1. Introducción y planteamiento del problema

La verificación de firmas es un componente crítico en la seguridad biométrica y a diferencia de la clasificación de imágenes tradicional, donde el objetivo es etiquetar una imagen en una categoría fija, la verificación de firmas requiere determinar si dos muestras pertenecen a la misma persona, incluso si el sistema nunca ha visto a esa persona antes, por lo que para resolver esto, nuestro proyecto está enfocado en tratar de resolver la problemática de las falsificaciones de firmas en distintos documentos sensibles y para eso utilizamos una arquitectura de red siamesa Nag (2022) con la paquetería PyTorch.

2. Dataset

2.1. Información y estructura

Se utilizó el dataset "*Signature Verification*" disponible en Kaggle, con el objetivo de que nuestra red neuronal sea capaz de diferenciar entre firmas verdaderas y falsas. Los datos son no estructurados (imágenes) y se organizan de la siguiente manera:

- Carpetas de imágenes: `./train` y `./test`.
- Metadatos: Archivos CSV (`test_data.csv`, `train_data.csv`) que contienen pares de imágenes y una etiqueta binaria (0 para firmas genuinas, 1 para las que no).

En las carpetas de *train* y *test* se tienen las imágenes correspondientes a las firmas recolectadas por el dataset en formato `.png`, mientras que en los archivos `.csv` se tienen los datos correspondientes a las imágenes en donde se tienen las etiquetas correspondientes si es que estas son verdaderas o falsas.

2.2. Preprocesamiento y limpieza

Durante la fase de preparación de datos, se enfrentaron y resolvieron dos problemas principales:

1. **Dataset duplicado:** Al descargar el dataset mediante `kagglehub`, notamos que las descargas se hacían en un lugar distinto y ocurría un problema, pues se duplicaba el dataset al moverlo, generando carpetas anidadas (`sign_data/sign_data/`), por lo que se implementó un script con la librería `shutil` para detectar y borrar el directorio duplicado si existía, solucionando problemas de rendimiento y espacio.
2. **Dataset sin validación:** El dataset original no tenía un conjunto de validación, por lo que partirlo aleatoriamente no estaría bien ya que como el dataset tenía muchísimas relaciones con otras imágenes entonces no podíamos tomar las carpetas de imágenes así nada más, además de que si tomábamos un

20% del csv de train para nuestra validación las firmas de una misma persona podrían aparecer tanto en entrenamiento como en validación y el modelo aprendería a reconocer a la persona, no a verificar la firma y esto produciría fugas de información y falsos sesgos en el modelo. Para solucionar esto, dado que el dataset está ordenado sistemáticamente por bloques de relaciones de individuos se realizó una partición basada en bloques completos de identidad, asegurando que las firmas de las personas del conjunto de validación fueran diferentes que las utilizadas para el entrenamiento.

2.3. Carga y aumento de datos

Se implementó una clase `SignatureDataset` heredando de `torch.utils.data.Dataset` así, con esta clase, podemos abrir las imágenes y etiquetas de cualquiera de nuestros dataframes de validación, entrenamiento y testeo. Siguiendo lo de [Dey et al. \(2017\)](#) en el que se utilizan redes siamesas (que aunque no las vimos en el curso las investigamos para ver por qué eran elegidas en el artículo), vimos que son ampliamente utilizadas junto con redes triplets para problemas de biometría. Principalmente de aquí tomamos las ideas de las capas que íbamos a utilizar así como un buen indicio de los hiperparámetros que podíamos llegar a utilizar.

A nosotras se nos ocurrió que utilizar `DataAugmentation` para variar las imágenes era una buena estrategia para darle aún más variedad al modelo a la hora de entrenar y asegurar que el modelo pudiera realmente aprender cuando se hiciera el entrenamiento:

- Redimensionamiento a (128, 256).
- Rotación aleatoria.
- Recortes aleatorios (Random Resized Crop).

3. Definición de la red neuronal

El paper [Dey et al. \(2017\)](#) nos dio un buen acercamiento y viendo un poco el panorama (y con el fine tuning que se nos permitió) decidimos usar parte de un modelo pre-entrenado el cual fue una `ResNet` que nos sirve principalmente para tener texturas y reconocimientos como ya definidos y que el aprendizaje del modelo pueda ser muchísimo más sencillo. Quitamos la última capa ya que la que tiene por defecto no nos sirve y tenemos que usar una que soporte las dimensiones e incrustaciones (embeddings) de nuestra imagen de 128 X 256. En estos artículos: [Srinidhi \(2021\)](#), [C. \(2025\)](#) es más claro como se puede adaptar un modelo pre-entrenado para ocuparlo en nuestro propios modelos. En estos casos se ocuparon para datasets de 10 clases, pero nosotras simplemente la ocupamos para firmas falsas y verdaderas. Además de que la Red Neuronal Siamesa nos permite aprender la similitud entre dos entradas.

3.1. Red Siamesa ResNet18

La arquitectura base es una **ResNet18** (Red Convolutacional Residual). Se realizaron las siguientes modificaciones:

1. Se eliminó la última capa completamente conectada original.
2. Se sustituyó por una nueva estructura secuencial (*Head*) diseñada para embeddings de 128×256 :
 - Capa Lineal ($512 \rightarrow 512$).
 - Función de activación **ReLU**.
 - Regularización **Dropout** (0.5) para evitar sobreajuste.
 - Capa Lineal ($512 \rightarrow 256$).
 - Función de activación **ReLU**.
 - Capa Lineal de salida ($256 \rightarrow 128$).

La salida del modelo pasa por una normalización L2 para mejorar el cálculo de distancias en la función de pérdida.

3.2. Función de Pérdida (Contrastive Loss)

Dado que el objetivo es acercar pares positivos y alejar pares negativos, se utilizó la función de pérdida contrastiva (*Contrastive Loss*), comúnmente usada para datos biométricos. Shankar (2022) esta función mide la similitud de los embeddings:

$$L = y \cdot D^2 + (1 - y) \cdot \max(0, m - D)^2 \quad (1)$$

Donde:

- D es la distancia euclidiana entre los embeddings.
- Y es la etiqueta (0: Genuina, 1: Falsa).
- m es el margen mínimo de separación.

Así, si el par tiene etiqueta 0, tratamos de acercar la distancia y si es 1, alejamos la distancia.

4. Hiperparámetros

Se utilizaron los siguientes hiperparámetros y configuraciones:

- **Optimizador:** SGD con Momentum (0.9) y para el decaimiento $1e^{-3}$.
- **Tasa de Aprendizaje (Learning Rate):** $1e^{-4}$.
- **Batch Size:** 32 (aprovechando GPU) o 16 (CPU).
- **Épocas:** 20, implementando *Early Stopping* con una tolerancia de 6 iteraciones para evitar overfitting, guardando siempre los pesos del modelo con la mejor pérdida en validación.

El entrenamiento se realizó minimizando la distancia entre firmas de la misma persona y maximizando la distancia entre firmas falsas o de personas diferentes.

5. Evaluación del modelo y resultados

5.1. Problema durante la evaluación

Durante la evaluación inicial, nos encontramos con una barrera en las métricas que no podíamos superar (F1-Score de 0.68%), sin importar las mejoras al modelo. Tras una revisión exhaustiva (donde incluso se consultaron LLMs sin éxito), se detectó un error lógico en el código de predicción.

La línea original:

```
prediccion = (distancia_euclidiana < umbral_distancia).int()
```

Estaba invirtiendo la lógica de las clases (considerando 1 como genuino en lugar de 0). Al corregir la condición a:

```
prediccion = (distancia_euclidiana > umbral_distancia).int()
```

Al invertir el signo, las predicciones se alinearon con la lógica de las etiquetas (donde 1 representa "diferente/falso"), corrigiendo el desempeño del modelo.

5.2. Resultados Finales

El modelo fue evaluado en el conjunto de prueba (`test_data`), utilizando un umbral de distancia de `**0.25**`. Las métricas obtenidas son altamente satisfactorias para un sistema biométrico:

Métrica	Valor
Accuracy	96.08 %
Precision	98.93 %
Recall	93.44 %
F1-Score	96.11 %

Cuadro 1: Resultados finales en el conjunto de prueba.

Estos resultados indican que el modelo es capaz de generalizar correctamente y distinguir con alta precisión entre firmas verdaderas y falsas, minimizando tanto los falsos positivos como los falsos negativos.

6. Modelo en producción

Usamos el framework Streamlit que ofrece Python para crear una aplicación web que ponga en producción nuestro modelo, para tener una idea de como pasar nuestro modelo a una aplicación de este tipo nos guiamos viendo videos, y en especial seguimos la estructura que se muestre en: [Wire \(2024\)](#).

Por lo que definimos un archivo `requirements.txt`, donde esta todo lo que se necesita instalar para poner en producción el modelo, y en `app.py` es donde definimos toda nuestra aplicación, sigue la siguiente estructura:

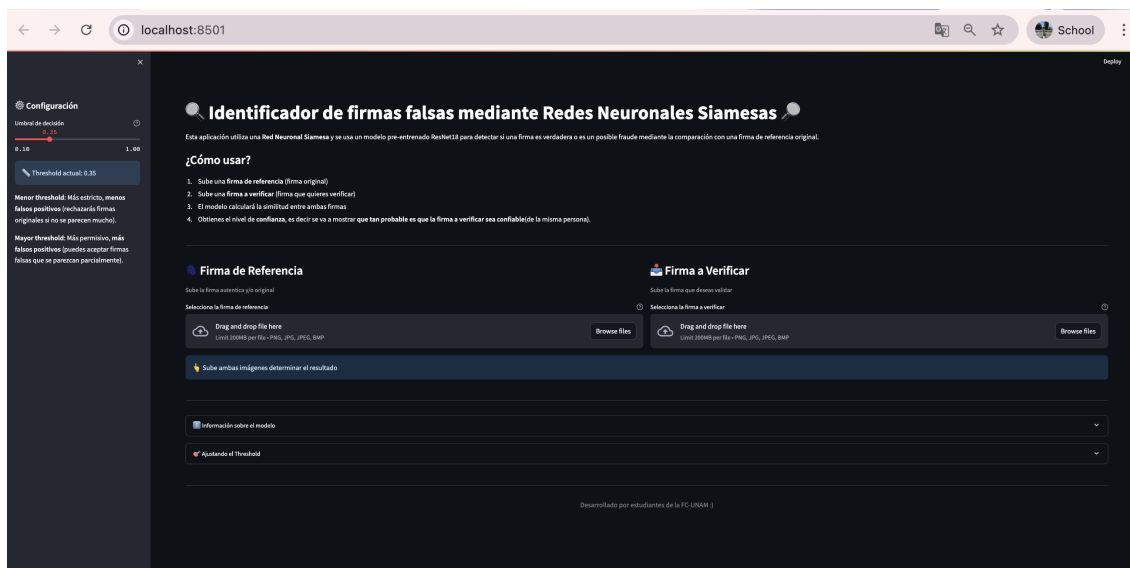
- Configuración inicial de la página (título, estilo, ect.).
- Cargar el modelo:
 - Copiamos la definición de nuestra red `SiamesCNN_RN18`.
 - Creamos una instancia de nuestra red la cual llamamos `modelo`.
 - Selecciona el dispositivo (cpu o gpu).
 - Cargamos los pesos de nuestro mejor entrenamiento en el `modelo`. Tales pesos se guardaron en `ProyectoRedesNeuronalesMejorM.ipynb` usando: `torch.save(modelo.state_dict(), 'mejor_modelo.entrenado.pt')` para guardar los parámetros aprendidos (los pesos y sesgos).
 - De tal forma que obtenemos nuestro `modelo` con el mejor resultado que nosotras obtuvimos, y ese es el que usamos para la producción.
- Definimos una función para transformar las imágenes, en especial para redimensionar las imagenes de entrada del mismo tamaño con el que el modelo trabajó: (128x256).
- Definimos una función para calcular la probabilidad de que una firma sea *confiable*, esta función calcula la *confianza* basado en los parámetros de *distancia* (distancia euclidiana entre los embeddings) y *threshold*. Para esta función buscamos la que mejor se adaptara en nuestro caso usamos un mapeo probabilístico con sigmoide.
- Definimos la interfaz de usuario primero con un template generado por una LLM (Gemini), y para cosas como cambios de color, o si queríamos otra columna o así nos ayudó mucho y ya con eso fuimos viendo que más cosas podíamos agregar, e igual ver videos sobre como se ve al poner en producción un modelo, nos dió una idea general de cómo queríamos o que tan interactiva queríamos nuestra aplicación web.

6.1. ¿Cómo probar?

Las instrucciones para poder acceder a la app son las siguientes:

- Clonar <https://github.com/xEriis/RedesNeuronales.git>
- Haber creado el environment (`conda env create -f environment.yml`).
- Una vez creado, activar con: `conda activate proyecto`
- Ejecutar `pip install -r requirements.txt` (las que ya tienes instaladas no se vuelve a instalar aja).
- Si no tienes cuenta, te pedirá crear una.
- Ejecutar `streamlit run app.py` a la altura de `./Proyecto`

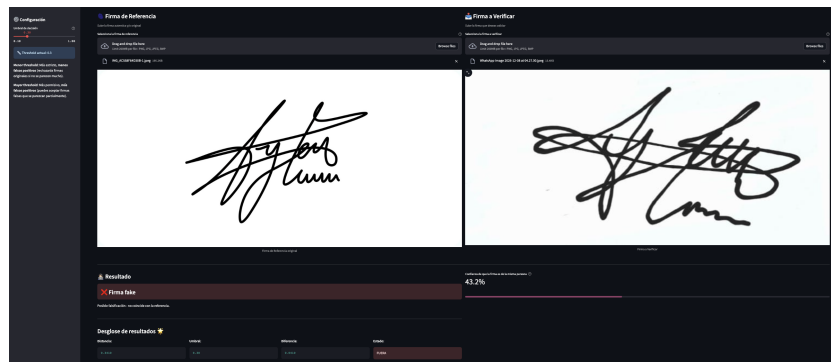
Ahora si podrás visualizar nuestra interfaz:)



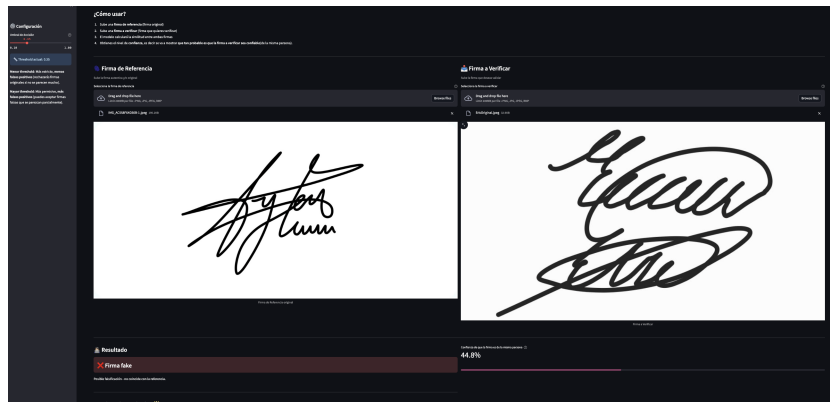
Adjuntamos algunas de nuestras pruebas, en nuestros casos notamos que poner un *threshold* entre 0,3 – 0,35 nos daba los mejores resultados.

- Caso: Firma 2 es falsa:

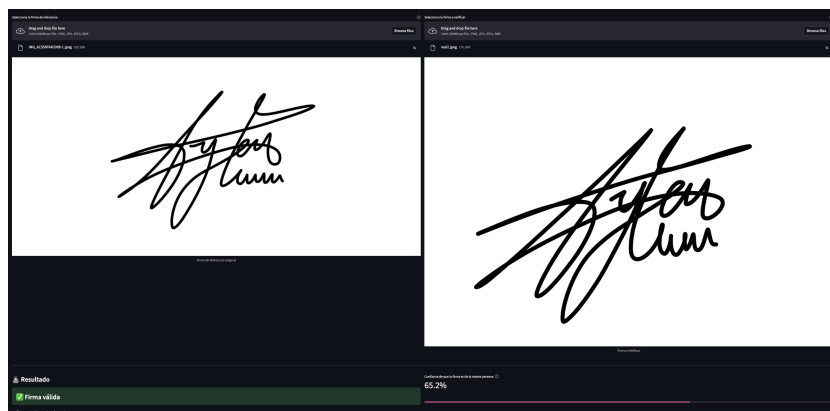
Nota: Puedes mover el umbral según el caso, por ejemplo si las dos firmas son parecidas pero tienes tus sospechas puedes poner un umbral un poco más bajo para ser estrictos.



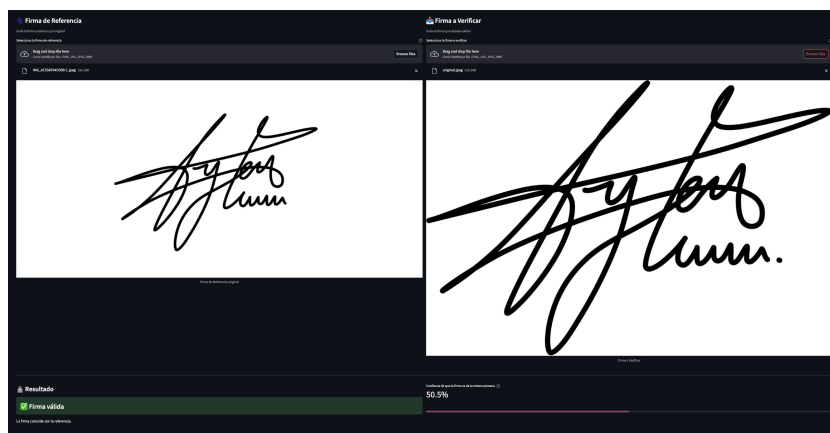
- Caso: Firma 2 es falsa: Por ejemplo aquí las dos firmas no se parecen entonces puedes incluso aumentar un poco el umbral y ver que aún así como que no las acepta como iguales.



- Caso: Las dos firmas son de la misma persona:



- Caso: Las dos firmas son de la misma persona:



7. Conclusiones

Este proyecto logró cumplir con el objetivo de diferenciar entre firmas genuinas y falsificaciones, ya que se implementó con éxito una arquitectura compleja basada en una ResNet18 de una Red Siamesa, la cual demostró ser adecuada para el procesamiento de datos no estructurados.

Además, se abordó y resolvió un problema crítico de fuga de información mediante el diseño de una estrategia de partición de datos basada en la identidad de las personas, asegurando una validación honesta. Por otro lado, la corrección de la lógica durante la etapa de evaluación resaltó la importancia de comprender la relación matemática entre la función de pérdida, donde el valor cero representa similitud, y el etiquetado del dataset. Por lo tanto, la exactitud obtenida confirma que el modelo propuesto es robusto y eficaz para tareas de verificación biométrica de firmas.

Referencias

- C., A. (2025). Custom Deep Learning Model with a ResNet18 Backbone for CIFAR-10 Image Classification. *Medium*. = <https://medium.com/@staytechrich/custom-deep-learning-model-with-a-resnet18-backbone-for-cifar-10-image-classification-4f844647e130>.
- Dey, S., Dutta, A., Toledo, Ghosh, Lladós, and Pal (2017). SigNet: Convolutional Siamese Network for Writer Independent Offline Signature Verification. *ELSEVIER*. <https://arxiv.org/pdf/1707.02131>.
- Nag, R. (2022). A Comprehensive Guide to Siamese Neural Networks. *Medium*. <https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513>.
- Shankar, G. (2022). Introduction to Contrastive Loss - Similarity Metric as an Objective Function. = <https://gowrishankar.info/blog/introduction-to-contrastive-loss-similarity-metric-as-an-objective-function/>.
- Srinidhi (2021). Implementing ResNet18 for Image Classification. *Kaggle*. <https://www.kaggle.com/code/ggsri123/implementing-resnet18-for-image-classification>.
- Wire, S. (2024). Machine learning model deployment using streamlit — python — streamlit. YouTube. Video.